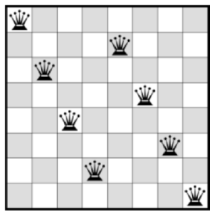
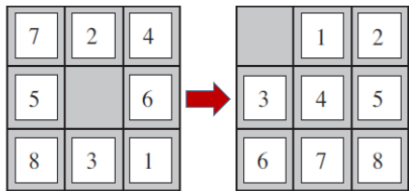


Lecture 3, 4

- reflex agents: 使用从状态到动作的映射 (查找)
- goal-based agents: 解决问题的代理或规划代理
 - 代理朝着**目标**努力工作。
 - 代理考虑**行动对未来状态**的影响, 这意味着他们的工作是识别导致目标的行动或一系列行动。
 - 形式化为对可能的解决方案的**搜索**。
- Problem solving as search
 - 通过以下方式定义问题:
 - 1) 制定目标
 - 2) 问题表述
 - 解决问题分为两个阶段:
 - 1) Search: “mental” or “offline” exploration of several possibilities
 - 2) 执行找到的解决方案
- Problem formulation
 - 问题定义
 - 初始状态: 智能体开始时的状态。
 - 棋盘中没有皇后。[8-queen problem]
 - 任意状态 (例如, 左侧的拼图配置)。[8-puzzle problem]
 - 在波士顿。[Routing problem]
- 状态: 从初始状态通过一系列动作可以到达的所有状态。(状态空间)
 - 棋盘上8个皇后的所有排列方式
 - 3x3格子中8个拼图的任意配置
 - 在城市中, 城市属于 ({New York, San Francisco, Denver,...})ss

- 动作：智能体可执行的动作。在状态 (s), (Actions(s)) 返回在状态 (s) 中可执行的一系列动作。（动作空间）
 - 将一个皇后添加到任意空格。
 - 向左、向右、向上或向下移动。
 - 步行至相邻城市。
- 转换模型：描述每个动作的结果, (Results(s, a))。
 - 更新后的棋盘。
 - 给定一个状态和一个动作, 返回结果状态。
 - 到达新的城市。
- 目标测试：判断给定状态是否为目标状态。
 - 在棋盘上放置 8 个皇后, 没有皇后在水平、垂直或对角线上攻击其他皇后。
 - 右侧的拼图配置?
 - 到达丹佛?
- 路径代价：给每条路径分配一个相对于性能度量的数值代价。
 - 移动次数
 - 路径长度（单位：公里）

• State space vs. search space（状态空间 vs 搜索空间）

- 状态空间（State space）：指物理上的配置，它表示问题中实体的实际状态或可能的排列。例如，拼图问题中的每个排列。
- 搜索空间（Search space）：指抽象的配置，由搜索树或搜索图表示所有可能解决方案的路径。例如，拼图问题中从初始状态到目标状态的搜索路径。
 - 搜索空间被分为三个区域：
 - 已探索区域（Explored）：也称为“已关闭列表”或“已访问集合”。表示已经检查过的节点。
 - 边界区域（Frontier）：也称为“准备列表”、“开放列表”或“边缘”。表示当前待处理的节点。
 - 未探索区域（Unexplored）：表示尚未检查的节点。
 - 搜索的核心是从未探索区域（3）移动节点到边界区域（2），再到已探索区域（1）。
 - 搜索策略的本质在于决定这些移动的顺序。
- 搜索树（Search tree）：表示动作的序列，包含以下元素：
 - 根节点（Root）：初始状态。
 - 分支（Branches）：动作。
 - 节点（Nodes）：动作产生的结果。节点包含以下信息：父节点、子节点、深度、路径代价以及与状态空间中的状态关联的信息。
- 扩展（Expand）：一个函数，用于给定一个节点，生成所有子节点。

• 搜索策略 (Search strategies)

- 定义：搜索策略通过选择节点扩展的顺序来定义。
- 评价维度：
 - 完备性 (Completeness)：如果解决方案存在，算法是否总能找到它？
 - 时间复杂度 (Time complexity)：生成或扩展的节点数量。
 - 空间复杂度 (Space complexity)：内存中存储的最大节点数量。
 - 最优性 (Optimality)：是否总能找到代价最低的解决方案？

• 搜索策略的具体测量

- 时间和空间复杂度的具体测量：
 - **b**：搜索树的最大分支因子（每个状态的动作数）。
 - **d**：解决方案的深度。
 - **m**：状态空间的最大深度（可能是无穷大，有时记作D）。

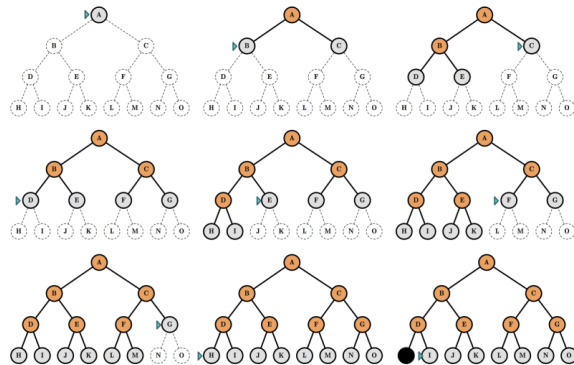
• 搜索种类：

• 无信息搜索 (Uninformed search)

- 特点：不使用领域知识。
- 策略：

• 广度优先搜索 (BFS)：扩展最浅的节点。

- BFS: Expand **shallowest** first.



```
function GRAPH-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE :

  initialize frontier with initialState
  explored = Set.new()

  while not frontier.isEmpty():
    state = frontier.remove()
    explored.add(state)

    if goalTest(state):
      return SUCCESS(state)

    for neighbor in state.neighbors():
      if neighbor not in frontier ∪ explored:
        frontier.add(neighbor)

  return FAILURE
```

```
function BREADTH-FIRST-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE :

  frontier = Queue.new(initialState)
  explored = Set.new()

  while not frontier.isEmpty():
    state = frontier.dequeue()
    explored.add(state)

    if goalTest(state):
      return SUCCESS(state)

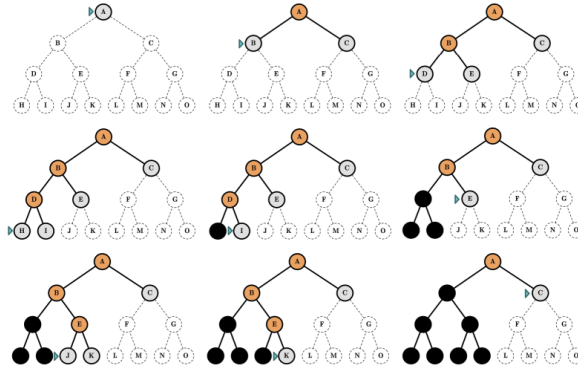
    for neighbor in state.neighbors():
      if neighbor not in frontier ∪ explored:
        frontier.enqueue(neighbor)

  return FAILURE
```

- 完备性 (Complete)：是 (若 b 有限)
- 时间复杂度 (Time)： $1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$
- 空间复杂度 (Space)： $O(b^d)$
- 最优性 (Optimal)：是 (若每步代价 = 1)

- 实现 (Implementation) : 边界结构: 先进先出 (FIFO, 队列)
- 深度优先搜索 (DFS) : 扩展最深的节点。
 - DFS: Expand **deepest** first.

Note: Once a node is expanded, it is removed from memory asap all its children are explored.



```
function BREADTH-FIRST-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE :
    frontier = Queue.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
      state = frontier.dequeue()
      explored.add(state)

      if goalTest(state):
        return SUCCESS(state)

      for neighbor in state.neighbors():
        if neighbor not in frontier ∪ explored:
          frontier.enqueue(neighbor)

    return FAILURE
```

```
function DEPTH-FIRST-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE :
    frontier = Stack.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
      state = frontier.pop()
      explored.add(state)

      if goalTest(state):
        return SUCCESS(state)

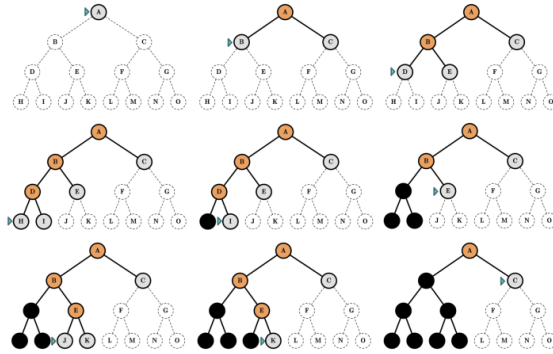
      for neighbor in state.neighbors():
        if neighbor not in frontier ∪ explored:
          frontier.push(neighbor)

    return FAILURE
```

- 完备性 (Complete) : 否 —— 在无限深度空间或存在环的空间会失败。
 - 可通过修改以避免沿路径重复状态来在有限空间中变为完备。
- 时间复杂度 (Time) : $1 + b + b^2 + b^3 + \dots + b^m = O(b^m)$
 - 如果 m 远大于 d 则很差, 但若解分布密集, 可能比 BFS 快得多。
- 空间复杂度 (Space) : $O(bm)$ —— 线性空间复杂度!
 - (只需存储从根到叶节点的一条路径, 以及该路径上每个节点的剩余未展开兄弟节点, 因此出现 m 因子。)
- 最优性 (Optimal) : 否
- 实现 (Implementation) : 边界结构: 后进先出 (LIFO, 栈)
- 深度限制搜索 (Depth-limited search, DLS) : 深度优先搜索, 设定深度限制。
 - 在深度限制 L 下的 DFS (位于层级 L 的节点没有后继节点)。
 - 选择一个深度上限 L , 用 DFS 在该深度限制下进行搜索。
- 迭代加深搜索 (Iterative-deepening search, IDS) : 深度限制搜索, 逐步增加限制深度。
 - 结合了 BFS 与 DFS 的优点。

Iterative deepening search (IDS)

- Limit = 3



- 思路：逐步增加搜索深度上限，直到到达最浅解的深度 d 为止。
- 应用：对每一轮使用具有增加上限的 DLS。
- 算法在找到解时停止，或当 DLS 返回失败（无解）时停止。
- 因为搜索树的大部分节点都位于底部，反复重新生成顶层节点并不是很浪费。
 - 例子：对深度上限在 0 到 3 之间逐轮搜索。
- 统一代价搜索（Uniform-cost search, UCS）：扩展代价最低的节点。
 - 修改 BFS：按代价而非深度优先级 → 展开具有最低路径代价 $g(n)$ 的节点 n 。
 - 按代价从低到高地进行扩展。

UCS: Pseudo-code

```
function BREADTH-FIRST-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE:
    frontier = Queue.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
      state = frontier.dequeue()
      explored.add(state)

      if goalTest(state):
        return SUCCESS(state)

      for neighbor in state.neighbors():
        if neighbor not in frontier ∪ explored:
          frontier.enqueue(neighbor)

    return FAILURE
```

```
function UNIFORM-COST-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE: /* Cost  $f(n) = g(n)$  */
    frontier = Heap.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
      state = frontier.deleteMin()
      explored.add(state)

      if goalTest(state):
        return SUCCESS(state)

      for neighbor in state.neighbors():
        if neighbor not in frontier ∪ explored:
          frontier.insert(neighbor)
        else if neighbor in frontier:
          frontier.decreaseKey(neighbor)

    return FAILURE
```

- 完备性（Complete）：是的，只要解的代价是有限的。
- 时间（Time）：
 - 设 C^* 为最优解的代价。
 - 每个动作的代价至少为 ϵ （代价下界）。
 - 有效深度大约为 C^*/ϵ （最便宜的解可能达到的深度）。
 - 时间复杂度约为 $O(b^{C^*/\epsilon})$ 。
- 空间（Space）：等于所有路径代价 $g \leq$ 最优解代价 的节点数， $O(b^{C^*/\epsilon})$

$\epsilon\}$ 。

- 最优性 (Optimal) : 是的 (会返回代价最小的解) 。
- 实现 (Implementation) : 边界 (frontier) 使用按路径代价 $g(n)$ 从小到大排序的队列 (优先队列/堆) 。
- 虽然 UCS 完备且最优, 但由于没有关于目标的任何信息, 它会向各个方向扩展搜索空间 (即按代价从低到高在整个状态空间中扩展) 。
- 有信息搜索 (Informed search) 【启发式搜索】
 - 使用启发函数 (heuristic) , 估计某个状态距离目标的远近
 - 策略:
 - 贪婪最佳优先搜索 (Greedy best-first search)
 - 评估函数 $h(n)$ (启发函数)
 - $h(n)$ 估计从节点 n 到目标的代价
 - 示例: $h_SLD(n)$ = 从 n 到 Sault Ste. Marie 的直线距离
 - 贪婪搜索展开看起来最接近目标的节点
 -

Greedy search: Pseudo-code

```
function GREEDY-BEST-FIRST-SEARCH(initialState, goalTest)
    returns SUCCESS or FAILURE : /* Cost  $f(n) = h(n)$  */

    frontier = Heap.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
        state = frontier.deleteMin()
        explored.add(state)

        if goalTest(state):
            return SUCCESS(state)

        for neighbor in state.neighbors():
            if neighbor not in frontier  $\cup$  explored:
                frontier.insert(neighbor)
            else if neighbor in frontier:
                frontier.decreaseKey(neighbor)

    return FAILURE
```

- 举例:

Greedy search example

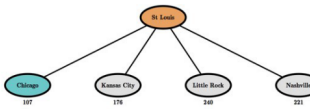
The initial state:



Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

Greedy search example

After expanding St Louis:

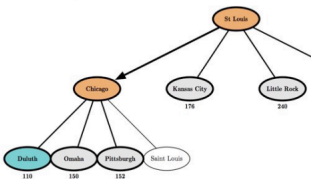


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

6

Greedy search example

After expanding Chicago:

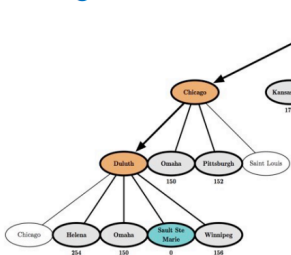


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

7

Greedy search example

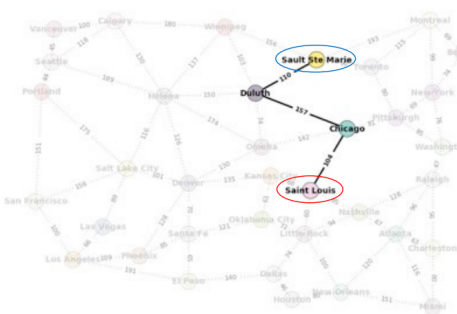
After expanding Duluth:



Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

Examples using the map (Greedy search)

Start: Saint Louis
Goal: Sault Ste Marie



- A* 搜索
 - 最小化对解的总代价的估计

- $f(n) = g(n) + h(n)$
 - $g(n)$: 到达节点 n 的实际代价
 - $h(n)$: 从节点 n 到目标的估计代价 (启发函数)
 - $f(n)$ 是经过 n 的最便宜解的估计代价

A* search: Pseudo-code

```
function A-STAR-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE : /* Cost  $f(n) = g(n) + h(n)$  */

  frontier = Heap.new(initialState) 最小值, 按  $f(n) = g(n) + h(n)$  排序
  explored = Set.new()

  while not frontier.isEmpty():
    state = frontier.deleteMin()
    explored.add(state)

    if goalTest(state):
      return SUCCESS(state)

    for neighbor in state.neighbors():
      if neighbor not in frontier ∪ explored:
        frontier.insert(neighbor)
      else if neighbor in frontier:
        frontier.decreaseKey(neighbor)

  return FAILURE
```

- 举例:

A* search example

The initial state:

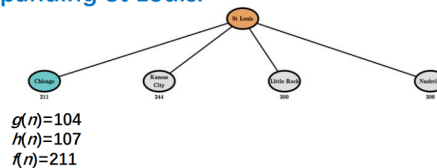


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

13

A* search example

After expanding St Louis:

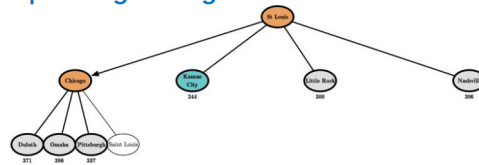


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

14

A* search example

After expanding Chicago:

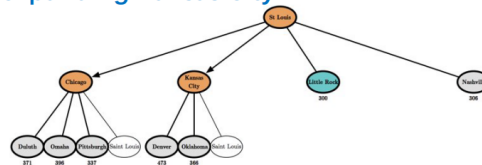


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

15

A* search example

After expanding Kansas City:

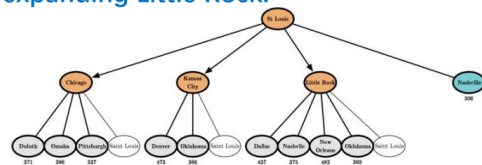


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

16

A* search example

After expanding Little Rock:

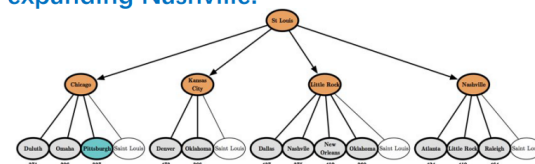


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

17

A* search example

After expanding Nashville:

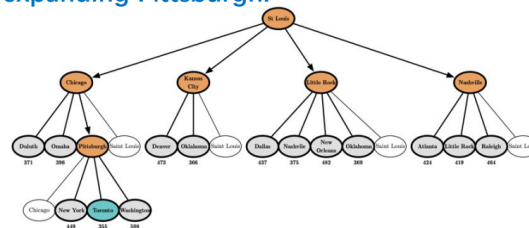


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

18

A* search example

After expanding Pittsburgh:

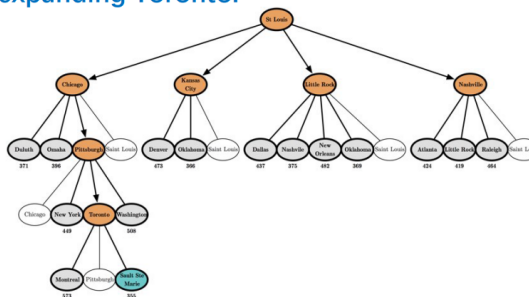


Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

19

A* search example

After expanding Toronto:



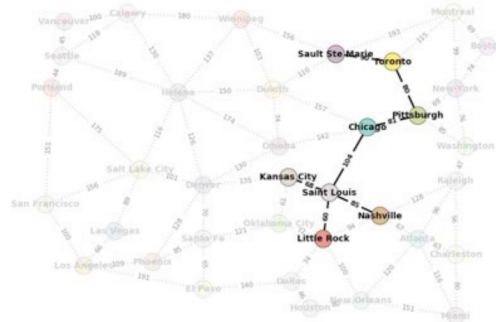
Atlanta	272
Boston	240
Calgary	334
Charleston	322
Chicago	107
Dallas	303
Denver	270
Duluth	110
El Paso	370
Helena	254
Houston	332
Kansas City	176
Las Vegas	418
Little Rock	240
Los Angeles	484
Miami	389
Montreal	193
Nashville	221
New Orleans	322
New York	195
Oklahoma City	237
Omaha	150
Phoenix	396
Pittsburgh	152
Portland	452
Raleigh	251
Saint Louis	180
Salt Lake City	344
San Francisco	499
Santa Fe	318
Sault Ste Marie	0
Seattle	434
Toronto	90
Vancouver	432
Washington	238
Winnipeg	156

20

Examples using the map (A* search)

Start: Saint Louis

Goal: Sault Ste Marie



21

- 完备性 (Complete) : 是的 (可以找到解, 若存在)
- 时间 (Time) : 指数级
- 空间 (Space) : 需将每个节点保存在内存中 (这是最大的问题)
- 最优性 (Optimal) : 是的 (在启发式可采的前提下)
- IDA* (迭代加深 A*)
- 可采 (admissible) 启发式:
 - 不高估到达目标的代价
 - 启发式 h 是可采的当且仅当: 对所有节点 n , $h(n) \leq h^*(n)$ 其中 h^* 表示从 n 到目标的真实最小代价。

- 在地图示例中使用的 h_SLD （直线距离启发式）是可采的，因为按定义它是两点之间的最短距离（直线），因此不会高估实际代价。
- A^* 的最优性：
 - 如果 $h(n)$ 是可采的，则在树搜索（tree search）下 A^* 是最优的。
 - 推理：
 - 设 G_o 是最优目标（代价最小的目标节点）， G_s 是某个次优目标（代价更大）。设 n 位于到 G_o 的最短路径上。
 - 因为对目标节点 $h(\text{目标})=0$ ，所以 $f(G_s)=g(G_s)$ ， $f(G_o)=g(G_o)$ ；由于 G_s 次优， $g(G_s) > g(G_o)$ ，因此 $f(G_s) > f(G_o) \dots(1)$ 。
 - 由于 h 是可采的， $h(n) \leq h^*(n)$ （ $h^*(n)$ 为从 n 到目标的真实最小代价），所以 $g(n)+h(n) \leq g(n)+h^*(n) = g(G_o) = f(G_o)$ ，因此 $f(n) \leq f(G_o) \dots(2)$ 。
 - 由 (1) 和 (2) 得到 $f(G_s) > f(n)$ ，因此在搜索过程中 A^* 不会选择 G_s （因为会优先扩展 f 值更小的节点 n 或者最终走向 G_o ），从而保证 A^* 找到的目标是最优的。
 - 结论：我们总是会朝向最优目标 G_o 而不是次优目标 G_s 。

- 启发式（Heuristics）

- 该解长 26 步。
- $h_1(n)$ = 错位方块的个数（misplaced tiles）= 8
- $h_2(n)$ = 总曼哈顿距离（横向和纵向距离之和）
- 起始状态中编号 1 到 8 的方块给出： $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$ ，这并不会高估真实解的代价。

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

2

- 启发式的搜索效率：
 - $h_mis(s)$ = 错位方块数，取值范围 $[0,8]$ ：可采（admissible）。
 - $h_1stp(s)$ = 达到目标配置所需的（1 步移动数）估计：可采。
 - 因为 $h_1stp(s) \geq h_mis(s)$ ，所以 $h_1stp(s)$ 比 $h_mis(s)$ 更“好”。
 - 支配性（Dominance）：
 - 对于可采的启发式 h_1 和 h_2 ，若对所有状态 s 都有 $h_1(s) \geq h_2(s)$ ，则 h_1 支配（dominates） h_2 ，并且在搜索中更高效。

- 定理：对于任意两个可采启发式 h_1 和 h_2 ，定义 $h(s) = \max\{h_1(s), h_2(s)\}$ ，则 $h(s)$ 也是可采的，且支配 h_1 与 h_2 。
- “更好”的启发式 = 支配 (dominance) = 更好的搜索效率。
- （注：支配性是一种形式化比较“更好”启发式的方法——在任意状态上估计值更高但不超过真实代价的启发式，一般会导致 A^* 等算法扩展更少的节点，从而更高效。）
- 更好的支配性
 - 问：从一组可采启发式 $h_1, \dots, h_m$ 中选哪一个？当彼此之间没有支配关系时该怎么办？
 - 答：定义 $h(s) = \max\{h_1(s), \dots, h_m(s)\}$ ，则 $h(s)$ 支配所有其它启发式（即在每个状态上估计更高且仍然可采），因此是更优的选择。

• 量化搜索效率 (Quantify Search Efficiency) :

- 有效分支因子 b^* :
 - 对于 A^* 找到的一个解，计算满足下式的 b^* :
 - $N = b^* + (b^*)^2 + \dots + (b^*)^d$
 - N : 求解过程中生成的节点总数
 - d : 解的深度 (解的步数)
 - 例如：若 A^* 在深度 5 处找到解且使用了 52 个节点，则 $b^* \approx 1.92$ 。
 - 目的：比较 h_1 与 h_2 在搜索效率上的差异。
 - 设置：生成 1200 个随机问题，解深 d 从 2 到 24 的集合；用 IDS（迭代加深搜索）以及用 A^* （分别配 h_1 和 h_2 ）来求解这些问题。
 - 说明：将 IDS 作为基准 (baseline) 。
 -

Empirical: Factor b^*

d	Search Cost (nodes generated)			Effective Branching Factor		
	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	—	539	113	—	1.44	1.23
16	—	1301	211	—	1.45	1.25
18	—	3056	363	—	1.46	1.26
20	—	7276	676	—	1.47	1.27
22	—	18094	1219	—	1.48	1.28
24	—	39135	1641	—	1.48	1.26

- $A^*(h_1)$ 与 $A^*(h_2)$ 在生成的节点数和有效分支因子上都远优于 IDS（基线）。IDS 的 b^* 大约在 2.7–2.9 左右，而 A^* 的 b^* 接近 1.3–1.5，说明启发式大幅减少了搜索规模。

- $A^*(h_2)$ (通常是更精细的启发式, 例如曼哈顿距离) 比 $A^*(h_1)$ (例如错位数) 表现更好: $A^*(h_2)$ 生成的节点远少于 $A^*(h_1)$, 其 b^* 更接近 1 (通常在 1.22–1.30 的范围), 意味着搜索非常高效。
- 好的启发式使得 b^* 接近 1
 - b^* 越接近 1 表示每层新增的节点非常少, 搜索非常高效, 能在合理的计算代价下解决更大的问题。
- b^* 用来量化启发式的搜索效率。 b^* 越大表示搜索扩展开得更快, 代价更高。
- 生成可采 (admissible) 的启发式:
 - 如何构造这样的“好”启发式?
 - 从放宽 (relaxed) 问题中生成
 - h_{mis} 与 h_{1stp} 来自哪里? 针对 8 拼图问题:
 - 真实规则: 一个方块只能移动到相邻的空格
 - 放宽规则 (relaxed rules): 由此得到的 h_{mis} 和 h_{1stp} 是可采的
 - R1: 一个方块可以移动到任意位置 $\Rightarrow h_{\text{mis}}(s) = \text{错位方块数}$
 - R2: 一个方块可以向任一方向移动一步, 且不受相邻方块被占用的限制 $\Rightarrow h_{\text{1stp}}(s) = \text{达到目标所需的 (一格移动) 步数 (即将每块看作可独立移动的一步计数)}$ 。
 - 对应于 R1、R2 的放宽问题更容易找到最优解。
 - 构造可采启发式的常用方法是把原问题的某些约束放宽, 计算放宽问题的最优代价作为原问题每个状态的启发值; 由于放宽问题的解代价 $\leq$ 原问题真实代价, 这样得到的启发式不会高估, 从而是可采的。
 - 从子问题生成 (启发式)
 - 子问题
 - 任务: 将方块 1、2、3、4 放回到它们的正确位置。
 - 放宽: 移动这些方块时忽略其它方块 (即不受其它方块约束)。
 - 理论: $\text{cost}^*(\text{子问题}) < \text{cost}^*(\text{原问题})$ 。
 - $\text{cost}^*(\text{子问题})$: 该子问题最优解的代价 (最小代价)。
 - 子问题把原问题分解, 只关心一小部分方块, 求解这类“更容易”的子问题可得出对原问题的下界, 从而得到可采启发值。
 - 可采的 $h_{\text{sub}}^*(s)$: 估计从状态 s 到子问题目标的代价。
 - 例如 $h_{\text{sub}}^*(1,2,3,4)(s)$ 表示解决「方块 1-2-3-4 子问题」的代价估计。

- 定理： $h_{\text{sub}}(s)$ 支配 $h_{1\text{stp}}(s)$
 - 其中 $h_{\text{sub}}(s) = \max\{h_{\text{sub}}^{(1,2,3,4)}(s), h_{\text{sub}}^{(2,3,4,5)}(s), \dots\}$
- 思路是：为不同的子问题分别计算最优代价（对每个子问题都是可采的），然后对这些子问题的估计取逐点最大值，得到的 h_{sub} 仍然可采且通常比简单的 1-step 启发式（ $h_{1\text{stp}}$ ）更强（支配它），因而能在搜索中更节省节点。
- 子问题不重叠
 - 问：若把子问题 (1-2-3-4) 和 (5-6-7-8) 的启发值相加，会得到一个可采的启发式吗？
 - 答：不会，因为它们总会共享某些移动
 - 问：如果不把那些共享的移动重复计数，情况如何？
 - 答：若满足 $h_{\text{sub}}^{(1,2,3,4)}(s) + h_{\text{sub}}^{(5,6,7,8)}(s) \leq c^*(s)$ （ $c^*(s)$ 为状态 s 的真实最优代价），则该和仍为可采启发式。

• 从经验中生成（启发式）

- “经验”构造方法
 - 对 8-拼图问题：
 - 求解大量 8-拼图以获得许多示例
 - 每个示例包含：解路径上的某个状态，以及从该状态到目标的实际代价
 - 这些示例就是我们针对该问题的“经验”。
- 从经验中学习启发式
 - 问：什么是好的经验特征？
 - 答：与预测状态到目标的代价相关的特征，例如：
 - $x_1(s)$ ：错位方块数、
 - $x_2(s)$ ：在当前状态相邻但在目标状态不相邻的方块对数
 - 问：如何从这些相关的经验特征中学习 h ？
 - 答：例如可以构建模型
 - $h(s) = w_1 \cdot x_1(s) + w_2 \cdot x_2(s)$
 - 其中 w_1 、 w_2 是需要从训练数据中学习的模型参数（可用神经网络、决策树等学习方法来训练）。

- Tree search:

```

function TREE-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE :

  initialize frontier with initialState

  while not frontier.isEmpty():
    state = frontier.remove()

    if goalTest(state):
      return SUCCESS(state)

    for neighbor in state.neighbors():
      frontier.add(neighbor)

  return FAILURE

```

- Graph Search:

```

function GRAPH-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE :

  initialize frontier with initialState
  explored = Set.new()

  while not frontier.isEmpty():
    state = frontier.remove()
    explored.add(state)

    if goalTest(state):
      return SUCCESS(state)

    for neighbor in state.neighbors():
      if neighbor not in frontier  $\cup$  explored:
        frontier.add(neighbor)

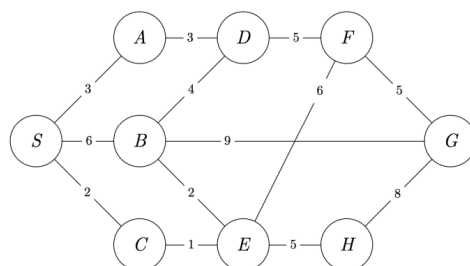
  return FAILURE

```

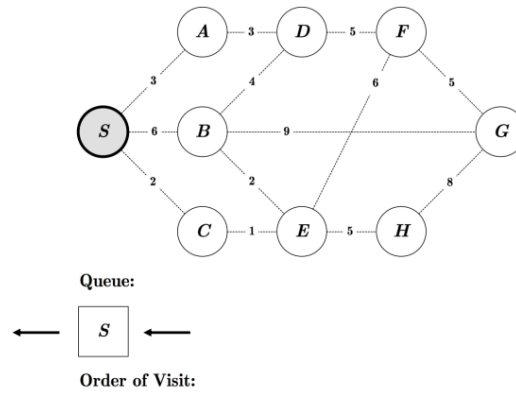
- 具体例子：

Toy Example

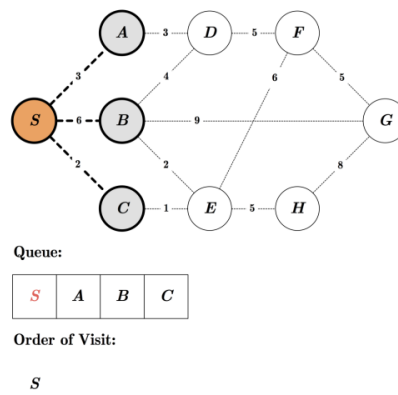
- Question:** What is the order of visits of the nodes and the path returned by BFS, DFS and UCS?



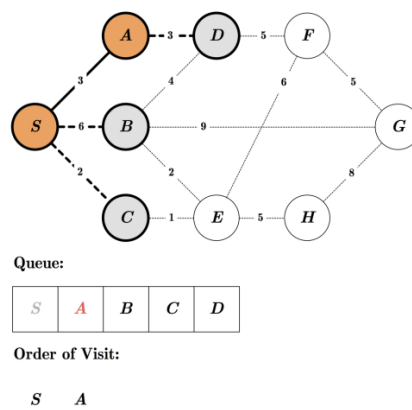
Toy Example: BFS



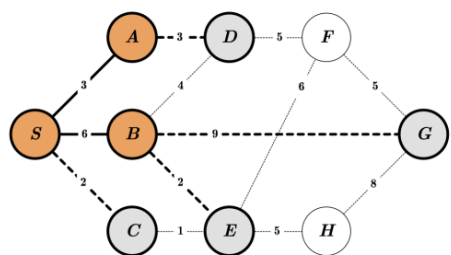
Toy Example: BFS



Toy Example: BFS



Toy Example: BFS



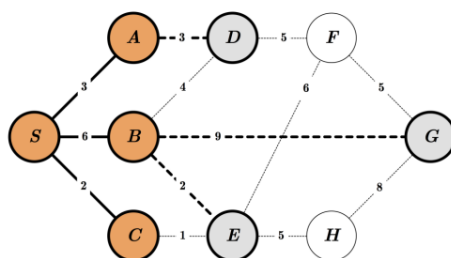
Queue:

S	A	B	C	D	E	G
---	---	---	---	---	---	---

Order of Visit:

S A B

Toy Example: BFS



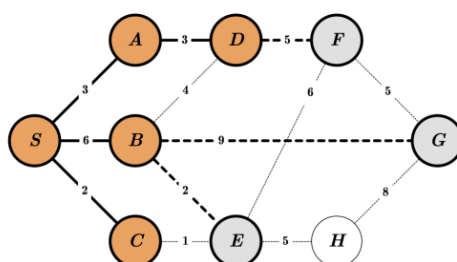
Queue:

S	A	B	C	D	E	G
---	---	---	---	---	---	---

Order of Visit:

S A B C

Toy Example: BFS



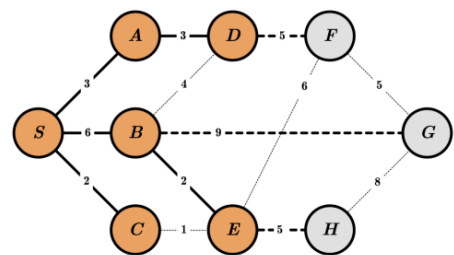
Queue:

S	A	B	C	D	E	G	F
---	---	---	---	---	---	---	---

Order of Visit:

S A B C D

Toy Example: BFS



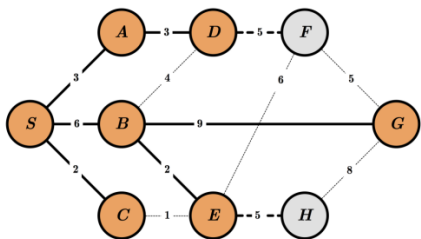
Queue:

S	A	B	C	D	E	F	G	H
---	---	---	---	---	---	---	---	---

Order of Visit:

S A B C D E

Toy Example: BFS



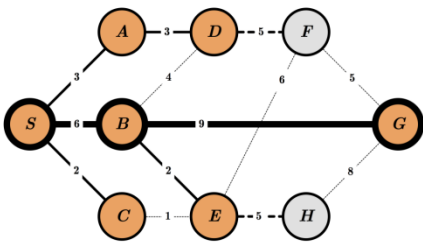
Queue:

S	A	B	C	D	E	F	G	H
---	---	---	---	---	---	---	---	---

Order of Visit:

S A B C D E G

Toy Example: BFS



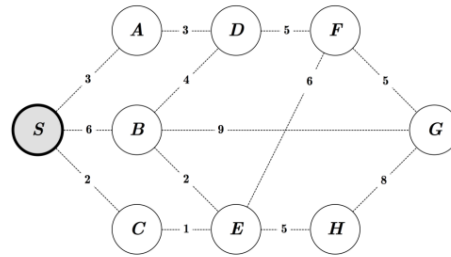
Queue:

S	A	B	C	D	E	G	F	H
---	---	---	---	---	---	---	---	---

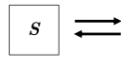
Order of Visit:

S A B C D E G

Toy Example: DFS

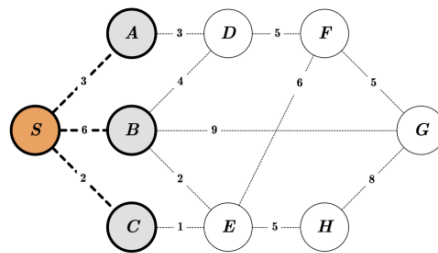


Stack:

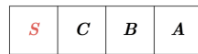


Order of Visit:

Toy Example: DFS



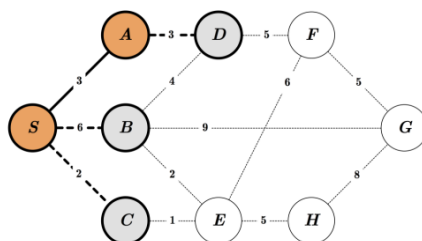
Stack:



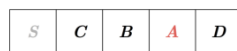
Order of Visit:

S

Toy Example: DFS



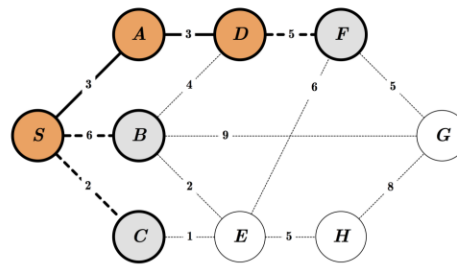
Stack:



Order of Visit:

S A

Toy Example: DFS



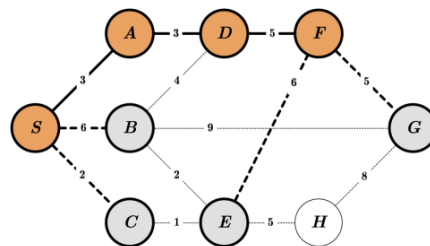
Stack:

S	C	B	A	D	F
-----	-----	-----	-----	-----	-----

Order of Visit:

$$S \quad A \quad D$$

Toy Example: DFS



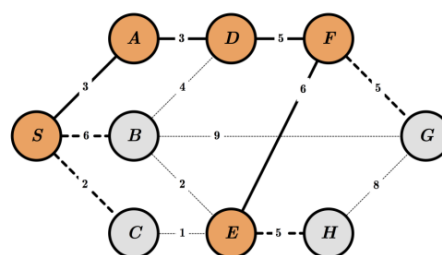
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>
----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

$$S \quad A \quad D \quad F$$

Toy Example: DFS



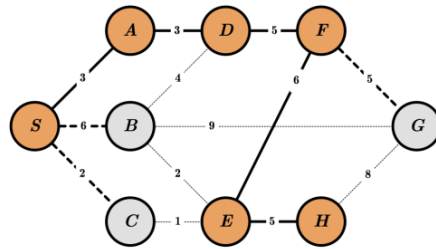
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S A D F E

Toy Example: DFS



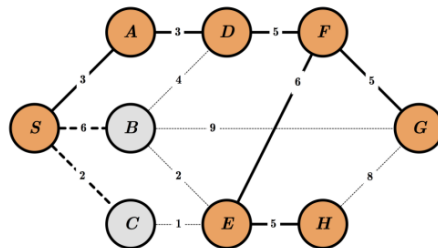
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E* *H*

Toy Example: DFS



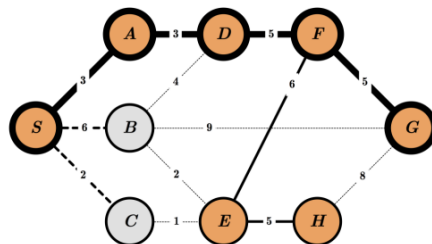
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E* *H* *G*

Toy Example: DFS



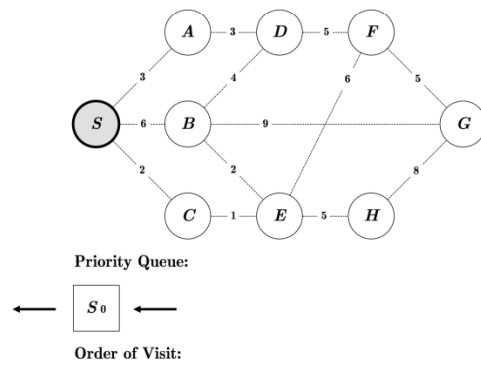
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

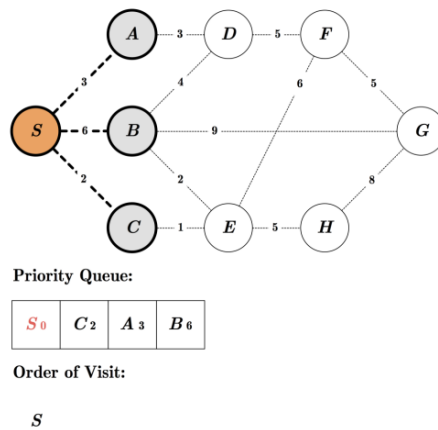
Order of Visit:

S *A* *D* *F* *E* *H* *G*

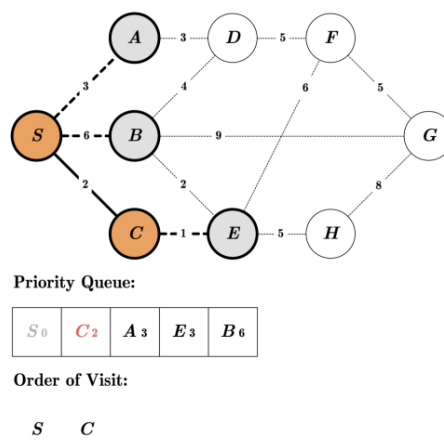
Toy Example: UCS



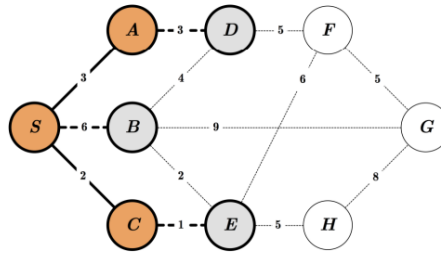
Toy Example: UCS



Toy Example: UCS



Toy Example: UCS



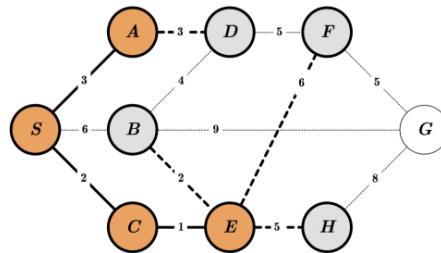
Priority Queue:

S_0	C_2	A_3	E_3	B_6	D_6
-------	-------	-------	-------	-------	-------

Order of Visit:

$S \quad C \quad A$

Toy Example: UCS



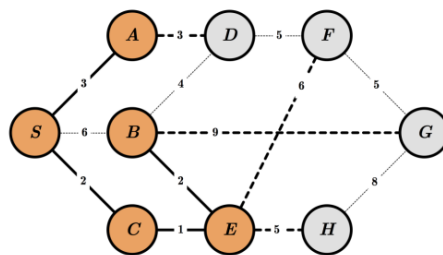
Priority Queue:

S_0	C_2	A_3	E_3	B_5	D_6	H_8	F_9
-------	-------	-------	-------	-------	-------	-------	-------

Order of Visit:

$S \quad C \quad A \quad E$

Toy Example: UCS



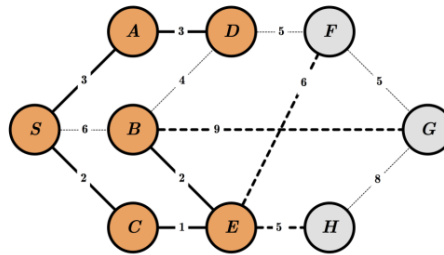
Priority Queue:

S_0	C_2	A_3	E_3	B_5	D_6	H_8	F_9	G_{14}
-------	-------	-------	-------	-------	-------	-------	-------	----------

Order of Visit:

$S \quad C \quad A \quad E \quad B$

Toy Example: UCS



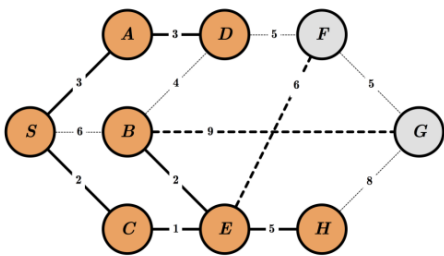
Priority Queue:

S_0	C_2	A_3	E_3	B_5	D_6	H_8	F_9	G_{14}
-------	-------	-------	-------	-------	-------	-------	-------	----------

Order of Visit:

$S \quad C \quad A \quad E \quad B \quad D$

Toy Example: UCS



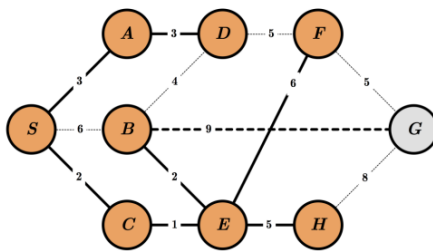
Priority Queue:

S_0	C_2	A_3	E_3	B_5	D_6	H_8	F_9	G_{14}
-------	-------	-------	-------	-------	-------	-------	-------	----------

Order of Visit:

$S \quad C \quad A \quad E \quad B \quad D \quad H$

Toy Example: UCS



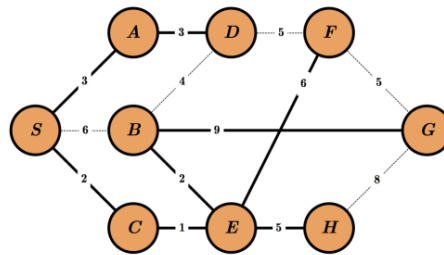
Priority Queue:

S_0	C_2	A_3	E_3	B_5	D_6	H_8	F_9	G_{14}
-------	-------	-------	-------	-------	-------	-------	-------	----------

Order of Visit:

$S \quad C \quad A \quad E \quad B \quad D \quad H \quad F$

Toy Example: UCS



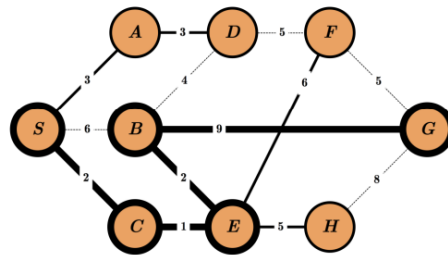
Priority Queue:

<i>S</i> ₀	<i>C</i> ₂	<i>A</i> ₃	<i>E</i> ₃	<i>B</i> ₅	<i>D</i> ₆	<i>H</i> ₈	<i>F</i> ₉	<i>G</i> ₁₄
-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	------------------------

Order of Visit:

S *C* *A* *E* *B* *D* *H* *F* *G*

Exercise: UCS



Priority Queue:

<i>S</i> ₀	<i>C</i> ₂	<i>A</i> ₃	<i>E</i> ₃	<i>B</i> ₅	<i>D</i> ₆	<i>H</i> ₈	<i>F</i> ₉	<i>G</i> ₁₄
-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	------------------------

Order of Visit:

S *C* *A* *E* *B* *D* *H* *F* *G*